# 第15课 串口控制

## 1.串口通信介绍及意义

控制一个 LED 的亮灭很容易，只需要设置其电压即可，但是如果我们想要控制由成百上千个 LED 组成的屏幕显示一个复杂的图案时，在程序中逐个设置 LED 的电压无疑会成为一件非常让人头疼的事情。

实际上，当我们使用一些复杂的硬件时，常常会需要发送一大串控制指令才能让它正常工作。针对这些需要一次发送大量数据的情况，串口通信诞生了，它能够一次发送很多数据，从而满足各种复杂的数据传输情况。和它类似的还有 IIC、SPI 通信等。

那什么是串口呢？串口也被称为串行通信端口。串口通信有多种标准，比如TTL、RS-232、RS-485，注意这些都是指电平标准（电信号）。通俗来讲，串口通信就是要实现不同电子设备之间的“**通讯对话**”。

串口通信是设备之间传输数据的一种方式，串口按位（bit）发送和接收数据。

## 2.串口控制总线舵机转动

我们机械臂会用到串行总线舵机，总线舵机是由串口指令进行控制的，串口波特率为115200。大家可以发送对应的指令给舵机，即可控制舵机转动，或者读取舵机信息。

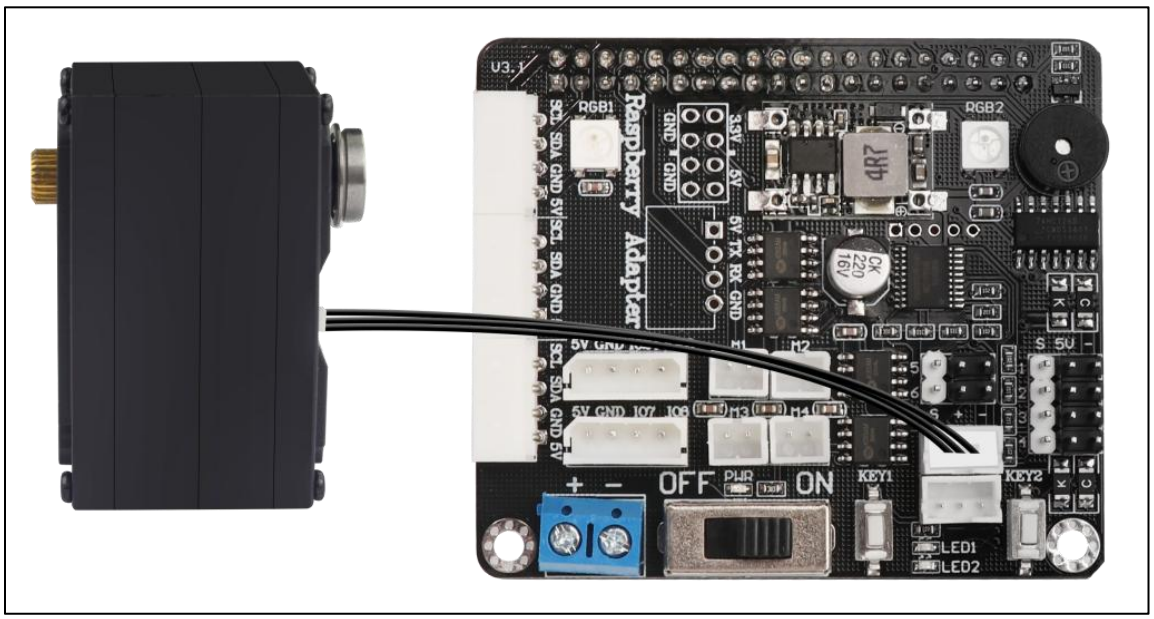
**特别注意**：在控制舵机之前，我们需要设置好舵机的各项参数及 ID。

同时，大家可以通过阅读我们提供的“**幻尔科技总线舵机通信协议**”文档，查看设置串口指令的具体方法。

## 2.1 准备工作

### 2.1.1 硬件接线

将总线舵机单独连接至树莓派扩展板上任意一个总线舵机接口上，这里以LX-15D舵机型号为例。




**特别注意：总线舵机线采用防反插设计，切勿硬塞。**

### 2.1.2 ID设置

我们以控制 ID6 舵机为例，可先通过 ArmPi FPV 上位机中的“**舵机调试工具**”进行设置。

1) 点击桌面上的“**PC_Software**”图标，打开上位机。

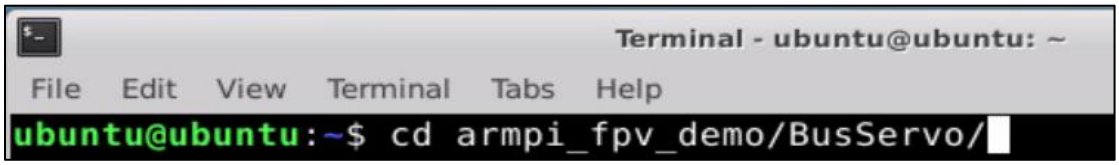
2) 点选切换到“**舵机调试工具**”菜单栏，单击“**读取**”，等待提示“**读取成功**”。

3) 在“**舵机ID**”区域，将ID选中，键盘输入6，单击“**设置**”，等待提示“**设置成功**”即可完成。

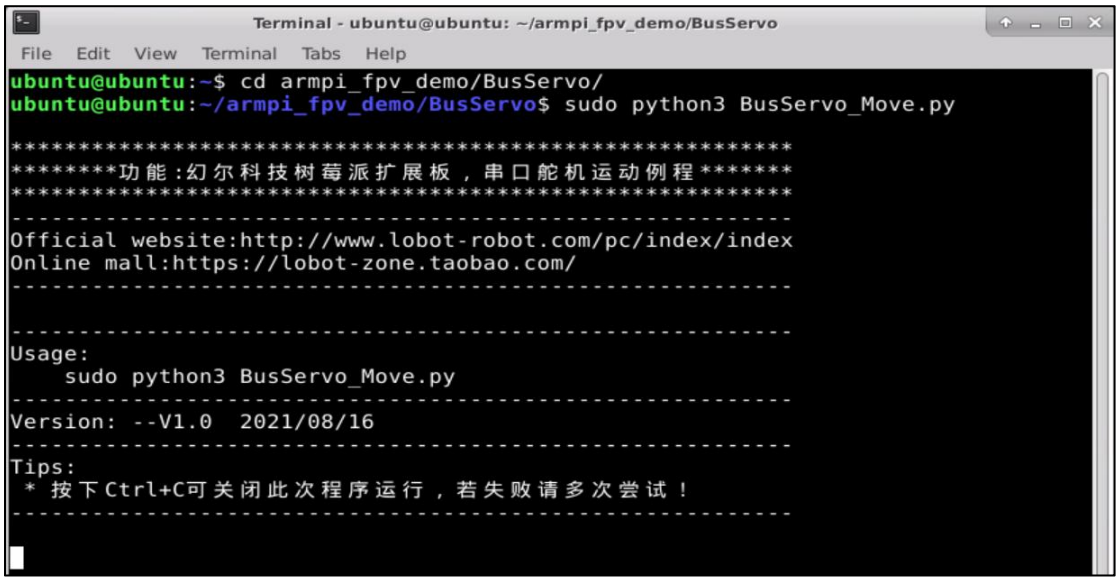
## 2.2 实验步骤

1) 打开 LX 终端。点击桌面左下角的“**Applications**”图标，然后选择“**Terminal Emulator**”打开 Ubuntu 的命令行终端。

2) 在打开的界面中，输入玩法程序所在目录的指令“**cd armpi_fpv_demo/BusServo/**”，按下回车键。




3) 然后输入运行玩法程序的指令“**sudo python3 BusServo_Move.py**”，按下回车键。




4) 如需关闭此程序，可按下“**Ctrl+C**”。

## 2.3 功能实现

程序运行后，总线舵机循环在800和200位置之间来回转动。

## 2.4 程序分析

代码中我们主要分为 3 个步骤控制总线舵机的运动。

◆ **导入扩展包**

要控制舵机，我们需要用到Board这个库文件，另外还需要日期和时间的模块。程序如下图：

```
import time
import Board
```

◆ **设置循环程序**

在导入扩展包后，便可控制舵机进行循环转动，其中括号内的参数分别代表：舵机 ID、转动位置、用时，我们设定在 800 和 200 位置之间进行，其中调用了 time 程序模块中的 sleep 程序，控制延时 0.5s。

程序如下图：

```
    Board.setBusServoPulse(6, 800, 1000) # 6号舵机转到800位置，用时1000ms
    time.sleep(0.5) # 延时0.5s

    Board.setBusServoPulse(6, 200, 1000) # 6号舵机转到200位置，用时1000ms
    time.sleep(0.5) # 延时0.5s
```